

ЛАБОРАТОРНАЯ РАБОТА №27 (2+2+2+2+2 БАЛЛА)

БИНАРНЫЕ ДЕРЕВЬЯ

Вариант	Задача 1	Задача 2	Задача 3	Задача 4	Задача 5
1	TreeWork25	TreeWork37	TreeWork65	TreeWork54	1
2	TreeWork26	TreeWork38	TreeWork66	TreeWork55	2
3	TreeWork27	TreeWork39	TreeWork67	TreeWork56	3
4	TreeWork28	TreeWork40	TreeWork68	TreeWork57	4
5	TreeWork29	TreeWork37	TreeWork41	TreeWork58	5
6	TreeWork25	TreeWork38	TreeWork42	TreeWork59	6
7	TreeWork26	TreeWork39	TreeWork65	TreeWork54	7
8	TreeWork27	TreeWork40	TreeWork66	TreeWork55	8
9	TreeWork28	TreeWork37	TreeWork67	TreeWork56	9
10	TreeWork29	TreeWork38	TreeWork68	TreeWork57	10
11	TreeWork25	TreeWork39	TreeWork41	TreeWork58	11
12	TreeWork26	TreeWork40	TreeWork42	TreeWork59	12
13	TreeWork27	TreeWork37	TreeWork65	TreeWork54	13
14	TreeWork28	TreeWork38	TreeWork66	TreeWork55	14
15	TreeWork29	TreeWork39	TreeWork67	TreeWork56	15

TREWORK

УДАЛЕНИЕ В ДЕРЕВЬЯХ ПОИСКА И ФОРМИРОВАНИЕ СТРУКТУРНОГО ОПИСАНИЯ БИНАРНЫХ ДЕРЕВЬЕВ

TreeWork25. Дано непустое дерево и корень дерева P_1 . Вывести строку с описанием исходного дерева в следующем формате: $\langle \text{дерево} \rangle ::= ((\langle \text{левое поддерево} \rangle) \langle \text{вершина} \rangle (\langle \text{правое поддерево} \rangle)) \mid ((\langle \text{левое поддерево} \rangle) \langle \text{вершина} \rangle) \mid (\langle \text{вершина} \rangle (\langle \text{правое поддерево} \rangle))$
 $\langle \text{вершина} \rangle ::= \langle \text{цифра} \rangle \langle \text{цифра} \rangle \mid \langle \text{цифра} \rangle \mid \langle \text{левое поддерево} \rangle ::= \langle \text{дерево} \rangle \mid \langle \text{правое поддерево} \rangle ::= \langle \text{дерево} \rangle$ Например, "(((1)2((3)4))5(6(7)))". Пробелы в результирующей строке отсутствуют, ссылки на пустые деревья никак не выводятся.

TreeWork26. Дано дерево поиска и корень дерева P_1 . Удалить в дереве корневую вершину. При замене содержимого удаляемой вершины использовать данные из ее левого поддерева. После удаления вывести строку с описанием исходного дерева в следующем формате: $\langle \text{дерево} \rangle ::= ((\langle \text{левое поддерево} \rangle) \langle \text{вершина} \rangle (\langle \text{правое поддерево} \rangle)) \mid ((\langle \text{левое поддерево} \rangle) \langle \text{вершина} \rangle) \mid (\langle \text{вершина} \rangle (\langle \text{правое поддерево} \rangle))$
 $\langle \text{вершина} \rangle ::= \langle \text{цифра} \rangle \langle \text{цифра} \rangle \mid \langle \text{цифра} \rangle \mid \langle \text{левое поддерево} \rangle ::= \langle \text{дерево} \rangle \mid \langle \text{правое поддерево} \rangle ::= \langle \text{дерево} \rangle$ Например, "(((1)2((3)4))5(6(7)))". Пробелы в результирующей строке отсутствуют, ссылки на пустые деревья никак не выводятся.

TreeWork27. Дано дерево поиска и корень дерева P_1 . Удалить в дереве корневую вершину. При замене содержимого удаляемой вершины использовать данные из ее правого поддерева. После удаления вывести строку с описанием исходного дерева в следующем формате: $\langle \text{дерево} \rangle ::= ((\langle \text{левое поддерево} \rangle) \langle \text{вершина} \rangle (\langle \text{правое поддерево} \rangle)) \mid ((\langle \text{левое поддерево} \rangle) \langle \text{вершина} \rangle) \mid (\langle \text{вершина} \rangle (\langle \text{правое поддерево} \rangle))$
 $\langle \text{вершина} \rangle ::= \langle \text{цифра} \rangle \langle \text{цифра} \rangle \mid \langle \text{цифра} \rangle \mid \langle \text{левое поддерево} \rangle ::= \langle \text{дерево} \rangle \mid \langle \text{правое поддерево} \rangle ::= \langle \text{дерево} \rangle$ Например, "(((1)2((3)4))5(6(7)))". Пробелы в результирующей строке отсутствуют, ссылки на пустые деревья никак не выводятся.

TreeWork28. Дано дерево поиска и корень дерева P_1 . Удалить в дереве вершину со значением K . При замене содержимого удаляемой вершины использовать данные из ее левого поддерева.

После удаления вывести строку с описанием исходного дерева в следующем формате: $\langle \text{дерево} \rangle ::= ((\langle \text{левое поддерево} \rangle) \langle \text{вершина} \rangle (\langle \text{правое поддерево} \rangle)) \mid ((\langle \text{левое поддерево} \rangle) \langle \text{вершина} \rangle) \mid (\langle \text{вершина} \rangle (\langle \text{правое поддерево} \rangle)) \langle \text{вершина} \rangle ::= \langle \text{цифра} \rangle \langle \text{цифра} \rangle \mid \langle \text{цифра} \rangle$ $\langle \text{левое поддерево} \rangle ::= \langle \text{дерево} \rangle$ $\langle \text{правое поддерево} \rangle ::= \langle \text{дерево} \rangle$ Например, "(((1)2((3)4))5(6(7)))". Пробелы в результирующей строке отсутствуют, ссылки на пустые деревья никак не выводятся.

TreeWork29. Дано дерево поиска и корень дерева P_1 . Удалить в дереве вершину со значением K . При замене содержимого удаляемой вершины использовать данные из ее правого поддерева. После удаления вывести строку с описанием исходного дерева в следующем формате: $\langle \text{дерево} \rangle ::= ((\langle \text{левое поддерево} \rangle) \langle \text{вершина} \rangle (\langle \text{правое поддерево} \rangle)) \mid ((\langle \text{левое поддерево} \rangle) \langle \text{вершина} \rangle) \mid (\langle \text{вершина} \rangle (\langle \text{правое поддерево} \rangle)) \langle \text{вершина} \rangle ::= \langle \text{цифра} \rangle \langle \text{цифра} \rangle \mid \langle \text{цифра} \rangle$ $\langle \text{левое поддерево} \rangle ::= \langle \text{дерево} \rangle$ $\langle \text{правое поддерево} \rangle ::= \langle \text{дерево} \rangle$ Например, "(((1)2((3)4))5(6(7)))". Пробелы в результирующей строке отсутствуют, ссылки на пустые деревья никак не выводятся.

РАБОТАЕМ С ДЕРЕВЬЯМИ ФОРМУЛ

TreeWork37. Дан корень дерева, хранящего арифметическое выражение. В выражении участвуют неотрицательные однозначные целые числа, знаки операций кодируются отрицательными числами: сложению соответствует -1, вычитанию -2, умножению -3, делению -4. Создать копию этого дерева и в созданной копии заменить все выражения вида: $1*a$, $a*1$, $a/1$ на a . Учесть, что в качестве a может быть выражение. Если в результате одного преобразования дерева вновь появляются выражения данного вида, их так же нужно преобразовывать и в результирующем дереве таких выражений быть не должно. Вывести указатель на корень дерева, полученного в результате, и полученное арифметическое выражение в виде строки символов.

TreeWork38. Дан корень дерева, хранящего арифметическое выражение. В выражении участвуют неотрицательные однозначные целые числа, знаки операций кодируются отрицательными числами: сложению соответствует -1, вычитанию -2, умножению -3, делению -4. Создать копию этого дерева и в созданной копии заменить все выражения вида: $0+a$, $a+0$, $a-0$ на a . Учесть, что в качестве a может быть выражение. Если в результате одного преобразования дерева вновь появляются выражения данного вида, их так же нужно преобразовывать и в результирующем дереве таких выражений быть не должно. Вывести указатель на корень дерева, полученного в результате, и полученное арифметическое выражение в виде строки символов.

TreeWork39. Дан корень дерева, хранящего арифметическое выражение. В выражении участвуют неотрицательные однозначные целые числа, знаки операций кодируются отрицательными числами: сложению соответствует -1, вычитанию -2, умножению -3, делению -4. Создать копию этого дерева и в созданной копии заменить все выражения вида: $0*a$, $a*0$, $0/a$, $a-a$ на 0 . Учесть, что в качестве a может быть выражение. Если в результате одного преобразования дерева вновь появляются выражения данного вида, их так же нужно преобразовывать и в результирующем дереве таких выражений быть не должно. Вывести указатель на корень дерева, полученного в результате, и полученное арифметическое выражение в виде строки символов.

TreeWork40. Дан корень дерева, хранящего арифметическое выражение. В выражении участвуют неотрицательные однозначные целые числа, знаки операций кодируются отрицательными числами: сложению соответствует -1, вычитанию -2, умножению -3, делению -4. Определить, возникнет ли ошибка "Деление на ноль!" при вычислении данного выражения. Вывести True, если это так, и False в противном случае.

БАЛАНСИРОВКА В БИНАРНЫХ ДЕРЕВЬЯХ ПОИСКА

TreeWork41. Дано дерево поиска и корень этого дерева. Если необходимо, привести дерево к АВЛ-сбалансированному виду, выполнив для этого LL-поворот. Известно, что требуется не более одного такого поворота.

TreeWork42. Дано дерево поиска и корень этого дерева. Если необходимо, привести дерево к АВЛ-сбалансированному виду, выполнив для этого RR-поворот. Известно, что требуется не более одного такого поворота.

БАЛАНСИРОВКА В БИНАРНЫХ ДЕРЕВЬЯХ ПОИСКА

TreeWork65. В первой строке текстового файла записаны целые числа, разделенные пробелами. Создать дерево поиска, последовательно включая в него перечисленные в файле числа. После этого необходимо, привести дерево к АВЛ-сбалансированному виду, выполнив для LL-поворот. Известно, что требуется не более одного такого поворота. Вывести корень полученного дерева.

TreeWork66. В первой строке текстового файла записаны целые числа, разделенные пробелами. Создать дерево поиска, последовательно включая в него перечисленные в файле числа. После этого необходимо, привести дерево к АВЛ-сбалансированному виду, выполнив для LR-поворот. Известно, что требуется не более одного такого поворота. Вывести корень полученного дерева.

TreeWork67. В первой строке текстового файла записаны целые числа, разделенные пробелами. Создать дерево поиска, последовательно включая в него перечисленные в файле числа. После этого необходимо, привести дерево к АВЛ-сбалансированному виду, выполнив для RL-поворот. Известно, что требуется не более одного такого поворота. Вывести корень полученного дерева.

TreeWork68. В первой строке текстового файла записаны целые числа, разделенные пробелами. Создать дерево поиска, последовательно включая в него перечисленные в файле числа. После этого необходимо, привести дерево к АВЛ-сбалансированному виду, выполнив для RR-поворот. Известно, что требуется не более одного такого поворота. Вывести корень полученного дерева.

ДЕРЕВЬЯ ПОИСКА, ОСНОВНЫЕ ОПЕРАЦИИ

TreeWork54. Дан текстовый файл, в первой строке которого записана последовательность неповторяющихся целых чисел. Числа разделены пробелами. Необходимо построить из этих чисел дерево поиска и вывести корень дерева P_1 .

TreeWork55. Дан текстовый файл, в первой строке которого записана последовательность неповторяющихся целых чисел. Числа разделены пробелами. Необходимо построить из этих чисел дерево поиска и вывести корень дерева, а также содержимое дерева, используя концевой обход.

TreeWork56. Дан текстовый файл, в первой строке которого записана последовательность неповторяющихся целых чисел. Числа разделены пробелами. Необходимо построить из этих чисел дерево поиска и вывести корень дерева, а также содержимое дерева, используя прямой обход.

TreeWork57. Дан текстовый файл, в первой строке которого записана последовательность неповторяющихся целых чисел. Числа разделены пробелами. Необходимо построить из этих чисел дерево поиска и вывести корень дерева, а также содержимое дерева, используя обратный обход.

TreeWork58. Дан текстовый файл, в первой строке которого записана последовательность неповторяющихся целых чисел. Числа разделены пробелами. Необходимо построить из этих чисел дерево поиска и вывести корень дерева, а также содержимое дерева так, чтобы числа были расположены в возрастающем порядке.

TreeWork59. Дан текстовый файл, в первой строке которого записана последовательность неповторяющихся целых чисел. Числа разделены пробелами. Необходимо построить из этих чисел дерево поиска и вывести корень дерева, а также содержимое дерева так, чтобы числа были расположены в убывающем порядке.

ЗАДАЧА 5

Реализовать определение класса (поля, свойства, конструкторы, перегрузку оператора вывода для вывода полей, заданный **метод** согласно варианту). Добавить перегруженные операторы. Протестировать все операторы и методы, включая конструкторы, **не забывайте проверять вводимые пользователем данные на корректность!**

№	Название класса	Поля	Метод	Перегружаемые операторы
1	Time	hours minutes	Добавление произвольного количества минут к объекту типа Time. Результат должен быть типа Time.	<u>Операции приведения типа:</u> int – результатом является количество часов (минуты отбрасываются); boolean – результатом является true, если часы и минуты не равны нулю, и false в противном случае. <u>Бинарные операции:</u> + Time t. Результат должен быть типа Time. – Time t. Результат должен быть типа Time.
2	Money	rubles kopeks	Добавление произвольного количества копеек к объекту типа Money. Результат должен быть типа Money.	<u>Операции приведения типа:</u> int – результатом является количество рублей (копейки отбрасываются); float – результатом является копейки, переведенные в рубли, целые рубли отбрасываются, результат <1. <u>Бинарные операции:</u> + Money m. Результат должен быть типа Money. – Money m. Результат должен быть типа Money.
3	Point	x y (координаты точки)	Вычислить расстояние от одной точки до другой. Результат должен быть типа float	<u>Операции приведения типа:</u> int – результатом является целая часть координаты x; float – результатом является координата y. <u>Бинарные операции:</u> + Point p – вычисляется расстояние до точки p. Результатом должно быть вещественное число. – целое число. Уменьшается координата x, результатом является объект типа Point.

4	RightTriangle	a b (длины катетов)	Вычислить площадь треугольника. Результат должен быть типа float.	<u>Операции приведения типа:</u> float – результатом является площадь треугольника, если треугольник существует и отрицательное число в противном случае; bool – результатом является true, если треугольник с такими длинами сторон существует и false в противном случае. <u>Бинарные операции:</u> <= RightTriangle t – сравнивает площади треугольников; >= RightTriangle t – сравнивает площади треугольников.
5	LineSegment	x y (координаты начала и конца отрезка на координатной прямой)	Определить, попадает ли заданное число в отрезок. Результат должен быть типа boolean.	<u>Операции приведения типа:</u> int – результатом является целая часть координаты x; float – результатом является координата y. <u>Бинарные операции:</u> + int d – координаты границ увеличиваются на число d); > int d – результат равен true, если целое число попадает в заданный отрезок и false – в противном случае.
6	QuadraticEquation	a b c (коэффициенты)	Вычисление вещественных корней квадратного уравнения. Результат должен быть списком величин типа float (в нем от 0 до 2-х элементов, в зависимости от количества корней).	<u>Операции приведения типа:</u> float – результатом является дискриминант уравнения. boolean – результатом является true, если корни существуют и false в противном случае. <u>Бинарные операции</u> == QuadraticEquation t — уравнения равны, если равны их коэффициенты; != QuadraticEquation t — уравнения не равны, если не равны их коэффициенты.

7	Time	hours minutes	Вычитание переменной типа Time (учесть, что возможен переход в предыдущие сутки). Результат должен быть типа Time.	<u>Операции приведения типа:</u> int – результатом является количество минут (время переводится в минуты); boolean – результатом является true, если часы и минуты не равны нулю, и false в противном случае. <u>Бинарные операции:</u> < Time t – время переводится в минуты, результатом является true, если количество минут в левом операнде меньше, чем количество минут в правом операнде, и false – в противном случае. > Time t — время переводится в минуты, результатом является true, если количество минут в левом операнде больше, чем количество минут в правом операнде, и false – в противном случае.
8	Money	rubles kopeks	Вычитание переменной типа Money (учесть, что денежная величина не может быть меньше 0). Результат должен быть типа Money.	<u>Операции приведения типа:</u> int – результатом является количество рублей (копейки отбрасываются); float – результатом является копейки, переведенные в рубли, целые рубли отбрасываются, результат <1. <u>Бинарные операции:</u> + положительное целое число, результат должен быть типа Money. - Money m результат должен быть типа Money. Результат не может быть отрицательным.
9	Point	x y (координаты точки)	Вычислить расстояние от точки до начала координат. Результат должен быть типа float.	<u>Операции приведения типа:</u> int – результатом является целая часть координаты x; float – результатом является координата y. <u>Бинарные операции:</u> + целое число – увеличивается координата x; – Point p – вычисляется расстояние до точки p, результатом должно быть вещественное число.

10	Triangle	a b c (длины сторон треугольник a)	Определить, существует ли треугольник с такими длинами сторон. Результат должен быть типа boolean.	<u>Операции приведения типа:</u> float – результатом является периметр треугольника; boolean – результатом является true, если треугольник с такими длинами сторон существует, и false в противном случае. <u>Бинарные операции:</u> < Triangle t - сравниваются площади треугольников; > Triangle t - сравниваются площади треугольников.
11	LineSegment	x y (координат ы начала и конца отрезка на координатн ой прямой)	Определить пересекаются ли заданные отрезки. Результат должен быть типа boolean.	<u>Операции приведения типа:</u> int – результатом является целая часть координаты x; float – результатом является координата y. <u>Бинарные операции:</u> - целое число – уменьшается координата y; < LineSegment d – результат равен true, если отрезки пересекаются, и false – в противном случае.
12	Time	hours minutes	Вычитание минут из объекта типа Time (учесть, что возможен переход в какие-либо предыдущие сутки). Результат должен быть типа Time.	<u>Операции приведения типа:</u> int – результатом является количество часов (минуты отбрасываются); boolean – результатом является true, если часы и минуты не равны нулю, и false в противном случае. <u>Бинарные операции:</u> == Time t результатом является true, если значение часов и минут левого операнда равно соответствующим значениям правого операнда, и false – в противном случае. != Time t результатом является true, если значение часов и минут левого операнда не равно соответствующим значениям правого операнда, и false – в противном случае.

13	Money	rubles kopeks	Вычитание копеек из объекта типа Money (учесть, что денежная величина не может быть меньше 0). Результат должен быть типа Money.	<u>Операции приведения типа:</u> int результатом является количество рублей (копейки отбрасываются); boolean результатом является true, если денежная сумма не равна 0. <u>Бинарные операции:</u> + положительное целое число. Число обозначает копейки - Money m вычитание денежных сумм
14	Time	hours minutes	Вычитание времени (величины типа Time) из объекта типа Time (учесть, что возможен переход в предыдущие сутки). Результат должен быть типа Time.	<u>Операции приведения типа:</u> int – результатом является количество часов (минуты отбрасываются); boolean – результатом является true, если часы и минуты не равны нулю, и false в противном случае. <u>Бинарные операции:</u> - положительное целое число минут к времени. + Time t – сложить два времени.
15	Money	rubles kopeks	Сложение двух объектов типа Money. Результат должен быть типа Money.	<u>Операции приведения типа:</u> int результатом является количество рублей (копейки отбрасываются); float результатом является копейки, переведенные в рубли, целые рубли отбрасываются, результат <1. <u>Бинарные операции:</u> - Money m – вычитание сумм + целое число – добавление копеек к денежной сумме